FinSight AI

Component Diagram Companion and Architecture Reading Guide

17 October 2026

Purpose: Explain what each FinSight AI component diagram is intended to communicate and how it maps to the Software Requirements Specification (SRS) and PostgreSQL schema.

Document role: Keep this file in the same project folder as the component-diagram images, the SRS PDF, and the database-schema reference. It is written so a developer or AI assistant can read it with the images and infer the intended system boundaries, responsibilities, data flows, persistence, and release scope.

Version: 1.0
Primary implementation focus: Release 1, source-backed financial analysis for US equities using historical daily data, with provider abstractions that can later support additional markets.

Quick navigation

- **Image 1:** Overall System Architecture - FinSight AI System Architecture(5).png
- **Image 2:** Market Data Ingestion and Data Quality - Market_data_ingestion_and_data_quality.png
- **Image 3:** Financial Research and Document Processing - FinSight AI Document Processing Architecture(1).png
- **Image 4:** Screening, Strategy and Backtesting - FinSight AI System Architecture Diagram(3).png
- **Image 5:** Quantitative Risk Analytics - FinSight AI Risk Analytics Architecture(2).png
- **Image 6:** Reporting and Saved Research - FinSight AI Reporting Architecture Diagram(1).png
- **Image 7:** Portfolio Management and Analytics - FinSight AI Portfolio Analytics Diagram(1).png
- **Image 8:** Paper Trading and Broker Integration, Release 2 - FinSight AI Trading Architecture Diagram(1).png
- **Additional coverage:** DCF, security/access control, Financial Data Explorer, and background jobs/observability are explained separately because the current image set does not contain a standalone component diagram for each one.

1. How to interpret this document and the diagrams

1.1 Source-of-truth order

When interpreting the project, use the following order of authority:

1. **The SRS defines the product requirements and nine application pages.** It is the authority for user-visible capabilities, access control, RAG behavior, market-data and data-explorer requirements, workflow execution, operations, settings, and account/workspace management.

2. **schema.sql defines the canonical core database tables, columns, primary keys, and foreign keys.** table_catalog.md explains the purpose of those tables. The database-schema PDF provides the broader ERD and schema reference.

3. **schema_optional_trading.sql defines the optional Release 2+ broker and trading extension.** Do not apply it as part of Release 1.

4. **The component diagrams explain logical components and intended data flows.** They are architectural views, not full ERDs. A box may represent a service, a user-interface feature, a background worker, an external provider, or a logical group of tables.

5. **This companion document explains the intent behind each image and identifies the canonical schema entities for that intent.** Where a diagram uses a shorthand or a generic field/table label, use the canonical name in the SQL schema rather than inventing a new table or field.

The schema package is a target-state design with 158 core tables and 15 optional trading tables. This is not a requirement to implement all 173 tables immediately. The DDL has been statically checked for table/FK target consistency, but this document does not claim that it has already been applied successfully to a running PostgreSQL instance.

1.2 Shared architecture decisions

- **Frontend:** Next.js and React, with dashboard, research, analysis, settings, and saved-work interfaces. Tailwind and charting/visualisation libraries are UI implementation choices.

- **Backend:** FastAPI provides authenticated APIs, validation, business logic, orchestration, data access, and background-job initiation.

- **Database:** PostgreSQL is the structured source of truth for identities, workspace permissions, instruments, prices, fundamentals, documents, analysis inputs/results, execution records, and saved reports.

- **RAG vector search:** The first-release design uses the pgvector extension inside the same PostgreSQL deployment. Treat the relational tables and vector index as distinct logical responsibilities, not separate physical databases.

- **LLM inference:** Ollama is the initial local/configurable inference runtime. Keep inference behind a provider interface so the model can be changed later.

- **Embeddings:** Embedding generation is a separate capability from chat completion. Model, dimension, chunking policy, and index versions must be compatible.

- **Data provenance:** Financial calculations and saved outputs should retain the input data_snapshot_id, applicable model/configuration versions, parameters, as-of date/time, and limitations so outputs can be explained and reproduced.

- **Security:** Authorization is enforced in backend services, not merely by hiding UI elements. RAG filters documents by user/workspace permissions before retrieved text is supplied to the model. Secrets are stored in a secrets manager; database fields hold references such as secret_ref, not plaintext credentials.

• **Background execution:** Long-running ingestion, parsing, embedding, indexing, risk, backtesting, and report jobs should be asynchronous and observable. PostgreSQL stores durable job and execution metadata; a queue/cache such as Redis may be used for dispatch but is not the authoritative record.

• **Market-data truthfulness:** Every financial value should retain source and time context. Label data as historical, end-of-day, delayed, cached, stale, demo, uploaded, derived, or otherwise as appropriate. Do not claim a real-time feed unless the configured provider and entitlement support it.

• **Release boundary:** Release 1 is read-only with respect to broker order execution. Paper trading and live broker execution are future capabilities, not dependencies for the initial product.

1.3 Nine application pages in the SRS

The component diagrams describe the backend and supporting services behind these pages:

• **P1 - Authentication and Access Control:** Sign-in, sessions, permissions, roles and protected operations. Connected to the overall architecture and all protected backend services, including document, report and future trading permissions.

• **P2 - Market and Portfolio Analytics Dashboard:** Market overview, charts, watchlists, portfolio performance, risk, alerts and freshness. Uses market ingestion, portfolio analytics, risk, reporting and background-job status.

• **P3 - AI Financial Analyst:** Natural-language questions, comparisons, supported calculations, source-backed explanations and workflows. Connects to overall architecture, document/RAG, market data, DCF, risk, strategy and reporting.

• **P4 - Financial Research and Knowledge Base:** Import, version, search, organise and retrieve financial documents. Connects to document processing, RAG and report citations.

• **P5 - Market Data and Financial Data Explorer:** Price/fundamental history, data cleaning, queries, datasets, transformations and exports. Connects to market ingestion, overall architecture and strategy/backtesting inputs.

• **P6 - Strategy, Screening, Signals and Workflow Studio:** Screen definitions, versioned strategies, signals, alerts and scheduled research workflows. Connects to screening/backtesting, job workers and approved AI tools.

• **P7 - Backtesting, Paper Trading and Execution Monitor:** Backtest results, trade-level analysis, comparisons and simulated positions; future execution history is conditional. Connects to screening/backtesting, risk, the future trading extension and jobs/observability.

• **P8 - Models, Market Data, Broker and RAG Settings:** Provider connections, refresh policies, model/embedding settings, risk and optional broker settings. Connects to overall architecture, ingestion, RAG, risk and Release 2 trading.

• **P9 - Account, Workspace and Saved Research:** Preferences, workspaces, saved screens/queries/strategies/reports/research and sharing. Connects to identity/security, reporting, portfolio, document library, strategy and workflow records.

The page labels identify product areas, not necessarily separate deployable microservices. Services can be implemented as modules within one FastAPI backend until there is a measured reason to split them.

2. Image 1 - Overall System Architecture

Image file: FinSight AI System Architecture(5).png
Primary purpose: Show the major layers of FinSight AI and how the web interface, API/backend services, LLM/RAG pipeline, PostgreSQL persistence, external providers, and background workers work together.

2.1 What the image is trying to communicate

This is the umbrella architecture. It explains where each major responsibility belongs and how requests cross boundaries. A user interacts with the web application; the backend authenticates and routes the request; domain services perform data retrieval or deterministic calculations; approved AI tools orchestrate selected functions; PostgreSQL stores authoritative data and execution records; external providers supply licensed data; and workers process long-running tasks.

The diagram is a logical architecture, not a deployment mandate. Boxes such as Market Data Service, DCF Valuation Service, and Reporting Service can initially be Python modules in a shared backend. Their separate boxes make ownership and dependencies explicit.

2.2 Frontend layer

The Next.js/React client represents the nine SRS pages: Authentication, Market/Portfolio Dashboard, AI Financial Analyst, Research/Knowledge Base, Data Explorer, Strategy/Workflow Studio, Backtesting/Monitor, Settings, and Account/Saved Research. UI components include charts, tables, reports, forms, and the chat experience. The frontend calls backend APIs over HTTPS/JSON; streaming may be used for AI responses when supported.

The frontend is not the authority for permission checks, financial calculations, secret handling, or data validation. It displays state returned by the backend and should surface timestamps, data mode, freshness, limitations, loading, failure, and completion states.

2.3 Backend/application layer

- **API Gateway (FastAPI):** Shared API entry point for routing, input validation, authentication/authorization enforcement, and rate limiting. It is a logical boundary; not necessarily a standalone reverse-proxy product.
- **Auth Service:** Handles users, roles, sessions, workspace access, and audit-related identity events. Backend endpoints must verify access independently of UI visibility.
- **Market Data Service:** Selects configured provider adapters and returns instrument metadata, prices, fundamentals, corporate actions, data snapshots, and freshness/quality information.
- **Portfolio & Analytics Service:** Manages portfolios, transactions, snapshots, watchlists, returns, performance, allocation, benchmark comparisons, and portfolio-derived outputs.
- **Research & Document Service:** Owns source/document metadata, upload/ingestion, versions, extraction status, chunks, indexing readiness, and search preparation.
- **AI Financial Analyst Service:** Handles conversation state, request classification, approved tool selection, response orchestration, and generation/result assembly. It must not let the LLM invent authoritative financial numbers.
- **DCF Valuation Service (Python):** Performs deterministic valuation calculations from financial history and explicit assumptions; see the scope note in Section 10 because there is no separate DCF image in the current set.
- **Quantitative Analysis Service:** Performs supported indicator/risk calculations and Monte Carlo or other quantitative analyses using persisted input versions.

• **Strategy & Backtesting Service:** Runs screeners, signals, versioned strategy logic, historical simulation, cost models, and performance evaluation.

• **Reporting Service:** Assembles saved analysis, charts/tables, source citations, input-snapshot lineage, and PDF/CSV artifacts.

• **Configuration Service:** Manages model/provider/RAG configuration, refresh and risk settings, health status, and optional future broker settings.

• **Tool Interface:** A controlled interface used by the AI orchestration layer to call allowlisted services. Every invocation needs authorization, validation, an explicit outcome, and an execution trace as appropriate. A natural-language request to analyse a strategy is not itself an order to trade.

2.4 AI/RAG layer and the intended query flow

The image's revised RAG flow is:

1. The Conversation Manager interprets the user query and applicable conversation context.

2. The RAG Retrieval Service embeds the query using a configured embedding model and identifies authorized search scope.

3. pgvector search retrieves semantically similar chunks; metadata/permission filters are enforced so inaccessible document content is not considered valid model context.

4. The pipeline returns ranked chunks and their source/document metadata.

5. The retrieved context and source references are assembled into the prompt for Ollama/LLM inference.

6. The Response Generator combines the model explanation with approved tool results, citations, warnings, and disclosures before returning the answer.

If the request is a deterministic financial question (for example, compute return, DCF, VaR, or a backtest), the analyst should call the corresponding approved backend calculation/tool, not rely on an LLM's approximate arithmetic. Retrieval and structured calculations are complementary: RAG supplies textual evidence; analytical services produce computed results.

2.5 Persistence and operational layer

PostgreSQL stores relational records and pgvector embeddings in the same deployment. Important logical domains include identity/workspaces; market/reference data and data_snapshots; ingestion batches and quality records; documents, versions, chunks and index metadata; conversations/messages and retrieval traces; portfolios and transactions; valuation and risk runs; screen/strategy/backtest data; reports/citations/sharing; and execution/job records.

Object/file storage holds original uploaded documents and generated report artifacts. The background job queue dispatches work to Python workers. Durable job, attempt, execution, step/event, and health information is stored in PostgreSQL tables such as background_jobs, job_attempts, execution_runs, execution_steps, execution_events, and service_health_checks.

External systems include market-data providers, approved financial-research/news sources, and optionally future brokers. Provider availability, licensing, data rights, API quota, and account entitlements are runtime conditions, not guarantees from the diagram.

2.6 Important boundaries

• PostgreSQL plus pgvector is the initial database design; do not infer a separate vector database deployment from the logical pgvector box.

• Market-data providers and financial-research sources are external inputs, not part of the application's trusted computation environment.

- DCF, risk, and backtesting calculations are backend responsibilities with provenance.
- Broker execution is a dashed/optional Release 2 extension.

---

3. Image 2 - Market Data Ingestion and Data Quality

Image file: Market_data_ingestion_and_data_quality.png
Primary purpose: Explain how external market-data responses become normalized, validated, snapshot-versioned records that the rest of the application can safely analyse.

3.1 Main pipeline

1. **Provider adapters** communicate with configured US-equity data APIs and translate provider-specific formats to an adapter contract. Alpha Vantage and Yahoo Finance are examples in the drawing, not an assumption that every endpoint, field, frequency, or redistribution right is available.

2. **Ingestion scheduler** applies refresh policies, incremental refresh, historical backfill, and queued ingestion tasks.

3. **Ingestion and raw capture** requests data, uses idempotency keys to control repeated work, and stores raw responses/hashes for audit and reprocessing.

4. **Normalize and map** resolves provider symbols to canonical instruments, maps company/ticker identities, normalizes units and timestamps, and structures prices, fundamentals, statements, and corporate actions.

5. **Validation and quality rules** checks schema/type validity, duplicates, missing intervals, invalid OHLC relationships, suspicious ranges, and consistency. Issues must be reported, not silently hidden.

6. **Snapshot manager** creates/updates a data snapshot, records batch outcomes, and promotes validated rows according to the ingestion policy.

7. **Market data store** persists normalized price_bars, financial_statements, financial_facts, corporate_actions, and the related canonical instrument/reference data.

8. **Data Quality Monitoring and Recovery** records issues and failures, supports retry/backfill/replay from captured raw records, and exposes operational state.

3.2 Canonical persistence mapping

Use these schema entities when interpreting the data cylinders and arrows:

- Provider and source configuration: source_registry, market_data_providers, provider_connections, provider_coverage, and refresh_policies.
- Snapshots and batches: data_snapshots and ingestion_batches.
- Raw provider records: raw_data_records.
- Quality rules/results: data_quality_rules and data_quality_issues.
- Canonical company/instrument data: companies, company_aliases, instruments, instrument_identifiers, instrument-symbol history, exchange/calendar, and classifications tables as applicable.
- Price/fundamental/event data: price_bars, financial_statements, financial_facts, financial_metric_definitions, and corporate_actions.

The intended foreign-key direction follows the schema: an ingestion batch refers to the relevant provider connection and data snapshot; raw records belong to an ingestion batch; quality issues refer to the

relevant rule, batch, and/or snapshot; data records link back to the relevant instrument, snapshot, and source as defined in SQL.

3.3 What downstream components receive

Market Data Explorer, dashboards, portfolio/risk analytics, valuation, screening, and backtesting consume normalized data rather than calling arbitrary provider APIs themselves. A response should preserve at least the as-of/observation timing, data frequency, source/provider lineage, adjustment/currency status, snapshot identity, and known quality limitations.

3.4 Release boundary and resilience

Release 1 focuses on historical daily equity data and supported fundamentals. Intraday or real-time data is conditional on configured provider support, rights, and entitlements. The retry/backfill path is for ingestion failures and missing history; it does not imply a broker order flow. Provider outages should not make unrelated pages unavailable. Repeated jobs must be idempotent to avoid duplicate active records.

---

4. Image 3 - Financial Research and Document Processing

Image file: FinSight AI Document Processing Architecture(1).png
Primary purpose: Describe the lifecycle from importing financial documents to storing searchable evidence that the AI Analyst can retrieve with access control and precise source citations.

4.1 Ingestion and sources

Inputs can include SEC/SEBI filings, annual reports, earnings materials, transcripts, research notes, and supported news/external sources. The ingestion API validates file type/size/content, captures available source URLs and metadata, identifies duplicates by content fingerprint, and checks user/workspace permission. Original files are retained in object/file storage according to the retention policy.

Source availability and licensing are conditional. Source registry records should preserve licensing state, attribution requirements, and permitted usage; an external source being reachable does not automatically grant the right to store, display, export, or redistribute its content.

4.2 Parsing, extraction, and OCR

The file parser handles supported formats such as PDF, TXT, DOCX, and HTML and performs format normalization/page segmentation. Content extraction captures text, headings, tables, units, reporting periods, dates, and key entities (company/ticker/table data). OCR is optional and used for scans or images whose text is not machine-readable. Extraction status/errors must be visible and retryable.

When practical, financial table extraction should preserve row/column relationships, headings, units, and reporting-period labels rather than flattening tables into unstructured prose.

4.3 Metadata enrichment and versioning

The enrichment stage associates a document with canonical company/instrument identities, document type, filing/publication/reporting date, source provider/URL, language, tags, and other supported metadata. Symbol/company resolution must avoid merging unrelated issuers or instruments.

The version manager records new versions, content hashes, supersession relationships, status and lineage. A revised filing or replacement document is a new version, not an invisible overwrite of the earlier evidence. The canonical document entities are research_sources, research_documents, and document_versions.

4.4 Chunking and embedding

The chunking service splits the extracted version into section-aware chunks and preserves page, section, table and text-location metadata. It supports overlapping chunks where configured. The embedding service computes vectors using the selected embedding model and stores the model/index relationship with the chunk.

The schema entities are document_chunks, chunk_embeddings, and knowledge_indexes. A chunk embedding is associated with both a stable document chunk and a knowledge index. The index records model/dimension and chunk-policy/index version; embeddings must match the selected dimensions and compatible configuration.

4.5 Indexing, retrieval, and permissions

The indexing service stores/updates vector embeddings and metadata, manages retries/partial failures, and ensures updates or deletions do not leave stale searchable content. Search-index maintenance can support vector and keyword/hybrid retrieval where enabled.

At query time, the retrieval service:

1. interprets the query and search scope;
2. determines the user's workspace and applicable access rights;
3. filters candidates using document/workspace permissions and metadata filters;
4. searches/ranks permitted chunks;
5. returns relevant chunk IDs, document/version IDs, location metadata and source references;
6. records retrieval diagnostics/evidence and provides only authorized context to the AI Analyst.

Relevant entities include document_access_grants, retrieval_runs, and retrieval_run_chunks in addition to chunk/index tables. Search permissions must be enforced before retrieved text is included in the LLM context, not after the model has seen it.

4.6 Document lifecycle, deletion and failure recovery

Updating a document may produce a new version and require re-indexing. Deletion must remove or deactivate searchable vectors and related metadata under the retention/backup policy. Queued work, chunks, embeddings, index status, and document-version history must be reconciled after partial failure. Jobs should be idempotent so retrying one document does not create duplicate active chunks or embeddings.

Operational persistence includes document_index_jobs and shared background_jobs/job_attempts/execution_runs records. Redis or another queue is optional dispatch/cache infrastructure, not the authoritative document-processing status store.

---

5. Image 4 - Screening, Strategy and Backtesting

Image file: FinSight AI System Architecture Diagram(3).png
Primary purpose: Show how structured market/fundamental data and user-defined rules become saved screens, versioned strategies, timestamped signals, historical simulations, performance metrics, and comparable reports.

5.1 Inputs

• Market history: OHLCV bars, corporate actions, reference/index/benchmark data, instrument metadata, and the selected data snapshot.

• Fundamentals: financial statements, normalized facts, metrics, classifications, and optional estimates where available.

• User/workspace definitions: saved screens, strategy rules/versions/variants, watchlists and explicitly selected instrument universes.

• Assumptions/configuration: market/universe choice, indicators, entry/exit rules, position sizing, risk controls, costs/fees, slippage/market impact, benchmark and scheduling choices.

Inputs flow from the data ingestion and application/configuration layers. A saved screen/backtest should not silently switch to current/latest data after it has been run.

5.2 Screening

The screening service evaluates an authorized instrument universe against fundamental/technical rules and ranks or returns matching instruments. Saved definitions can be reused; each screen execution retains its status, parameters, and input data_snapshot_id.

Canonical persistence is saved_screens, screen_runs, and screen_results. screen_runs references the saved screen, an execution run, and a data snapshot. screen_results is keyed by (screen_run_id, instrument_id) and retains matched conditions and metrics. These relationships make a result set reproducible and traceable to its screen and data inputs.

5.3 Strategy management and indicators

Strategies define reusable strategy identity/ownership. Immutable strategy_versions hold the validated strategy definition and entry, exit, sizing, and risk rules. strategy_variants allows named parameter overrides for repeatable comparison. The version is the execution unit; editing a strategy creates a new version rather than mutating what an earlier run used.

Indicators are configured using technical_indicator_definitions, indicator_profiles, and indicator_profile_items. The current schema does not define a generic persistent indicator_values table. Indicator values can be computed on demand/cached by the implementation, and relevant observations can be persisted as evidence for generated signals. Do not infer that indicator_values exists merely because a diagram describes an Indicator Calculation Service.

5.4 Signal generation and lifecycle

The signal service combines technical/fundamental indicators and strategy rules to generate buy/sell/hold or other supported research signals. It records the applicable instrument, strategy version/screen run where relevant, data_snapshot_id, input evidence, triggering rule, generation time and current status.

Canonical persistence includes signals, signal_observations, and signal_status_history. Signals are analytical outputs, not instructions to place a real order. The SRS explicitly separates generating an analysis/signal/strategy from submitting a real-money order.

5.5 Backtesting engine

The backtest engine evaluates a strategy version/variant over a historical date range. It simulates portfolio decisions and positions, rebalancing, sizing, transaction costs, slippage and market impact, and benchmark comparison. It should document supported integrity checks for look-ahead bias, corporate actions, missing data and other dataset limitations; support out-of-sample or walk-forward evaluation when implemented; and report unsupported assumptions instead of disguising them.

A backtest_runs record should retain at minimum the strategy_version_id, optional strategy_variant_id, data_snapshot_id, optional cost_model_version_id, date range, interval, initial capital, currency, benchmark instrument, execution assumptions, integrity checks, out-of-sample configuration, status and data limitations. It is associated with the durable execution_run_id.

The result tables include backtest_trades, backtest_metrics, and backtest_equity_points. The equity table records the portfolio/benchmark curve, period return, drawdown and exposure series. Saved report/export artifacts use the general reports model and report/execution links, not an assumed standalone backtest_reports table.

5.6 Performance evaluation and orchestration

Performance evaluation derives supported statistics such as CAGR, Sharpe/Sortino, drawdown/Calmar, win rate/profit factor, trade-level summaries and benchmark comparison from backtest outputs. Visualization/reporting reads the stored result series and trade/metric records. Scheduled screening, indicator calculation and backtesting use background workers, execution tracking, retries and notifications.

---

6. Image 5 - Quantitative Risk Analytics

Image file: FinSight AI Risk Analytics Architecture(2).png
Primary purpose: Describe the input preparation and parallel analytical methods used to quantify portfolio downside risk, volatility, drawdowns, simulated outcomes, and scenario stress impacts.

6.1 Inputs and preparation

The service consumes price bars/returns, corporate-action-aware adjustments, instrument data, a selected portfolio snapshot and its positions/weights, benchmark data, and risk configuration. Preparation aligns time series, selects the analysis window/frequency and universe, handles missing observations and outliers, and documents constraints. Risk calculations must not proceed as if missing data were complete; limitations are surfaced in the run output.

The portfolio snapshot and market-data snapshot are separate inputs. A risk run records both relationships as applicable so it is possible to identify which holdings and which prices were used.

6.2 Shared preparation and covariance estimation

Data Preparation & Return Calculation builds aligned return series and weights. Covariance Estimation Service estimates the covariance/correlation matrix using the selected supported method (for example sample covariance, shrinkage, or EWMA). These shared outputs support methods that require them; the diagram's VaR/CVaR, drawdown, Monte Carlo and stress-testing boxes are separate analytical branches, not a mandatory VaR-then-drawdown-then-simulation sequence.

6.3 Risk calculation methods

- **VaR/CVaR:** Historical and parametric methods, with Monte Carlo VaR/CVaR where supported, at selected confidence levels and time horizons. Outputs must retain method, assumptions, confidence level and limitations.
- **Drawdown analysis:** Cumulative returns, high-water marks, peak-to-trough drawdown, drawdown duration/recovery and rolling risk series.

• **Monte Carlo simulation:** Generates paths from a documented method/configuration; may use GBM, bootstrap, or other explicitly supported approaches. The run stores a random_seed and simulation count when applicable so the result can be reproduced.

• **Stress testing:** Applies named historical, factor, sector or custom shocks. A stress scenario definition is distinct from the result produced when a particular risk run applies it.

• **Aggregation/reporting:** Consolidates metrics, benchmark comparisons, time series, contributions and charts. It should preserve per-run status, limitations, and input provenance.

6.4 Canonical persistence

• risk_runs: run configuration and lineage, including execution_run_id, workspace/owner, portfolio and portfolio snapshot, risk model/config versions, data snapshot, benchmark, date range, frequency, confidence, annualization, optional simulation count/seed/covariance method, configuration snapshot, status and data limitations.

• risk_run_positions: frozen positions/weights used for the risk calculation.

• risk_metric_results: named risk outputs, composite key (risk_run_id, metric_key).

• risk_covariance_pairs: pairwise covariance/correlation entries, composite key (risk_run_id, instrument_a_id, instrument_b_id).

• risk_time_series: returns, portfolio value, drawdowns/high-water marks/rolling volatility by observation date, keyed by risk run and date.

• risk_distribution_points: persisted distribution/quantile points where retained.

• risk_simulation_points: retained Monte Carlo sample paths/steps where enabled.

• stress_scenarios and risk_stress_results: configured shock definition and run-specific result, respectively.

The exact column names and composite primary keys above come from the schema. Diagram database cylinders are compact summaries; they should not be treated as permission to invent standalone result-ID fields where the schema uses composite keys.

6.5 Run integrity and monitoring

A risk run should be attached to an execution record and preserve model/config version, source snapshot, frozen positions, methods, assumptions, random seed if used, status, and data limitations. The UI should display data-quality/freshness warnings and calculation status. Risk thresholds may trigger alerts, but those alerts are not an order-execution action.

---

7. Image 6 - Reporting and Saved Research

Image file: FinSight AI Reporting Architecture Diagram(1).png
Primary purpose: Show how validated outputs from financial-analysis services are assembled into persistent, source-backed reports that can be reopened, exported, and shared with explicit permissions.

7.1 Frontend responsibilities

• **Report Builder UI:** Choose analysis type (valuation, risk, portfolio, backtest, screening, or custom), sections, date range, data snapshots, and parameters; preview before generation.

• **Charts & Tables:** Present analytical data as charts, drill-downs and custom tables.

• **Saved Research Library:** Search/filter/reopen reports, organise saved items and manage accessible personal/workspace collections.

• **Export & Share Panel:** Request PDF/CSV generation, inspect status/history, download permitted files and manage sharing.

• **Report configuration/UI state:** Retain selected sources, filters, parameters, date ranges, layout, template and draft state as applicable.

7.2 Backend flow

1. The Reporting API authenticates/authorizes the request, validates the requested analysis inputs and snapshots, then creates/schedules report generation.

2. Report Assembly retrieves validated valuation/risk/portfolio/backtest/screen outputs from domain services and composes narrative, charts and tables.

3. Citation & Provenance links textual evidence and structured financial facts to their exact source/version/section/URL when available. It also records which data_snapshot_id values were used by the report.

4. Export Service renders PDF/CSV artifacts, stores them in file/object storage, and updates report artifact/status metadata.

5. Background jobs and execution tracking handle long-running export, failure/retry, status and notification.

The report should distinguish structured analysis inputs from explanatory source citations. A report can cite a document chunk/fact and also retain the snapshot of market/financial data used in the calculation; one does not replace the other.

7.3 Canonical persistence

• reports: report identity, owner/workspace, optional execution_run_id, type, title, summary/content, artifact URI/MIME type, as-of timestamp, status and created/updated timestamps.

• report_data_snapshots: report-to-input snapshot lineage, composite key (report_id, data_snapshot_id), plus usage purpose.

• report_citations: source-backed references to document versions/chunks, financial facts, source registry items or source URLs, with location/access metadata where available.

• report_access_grants: explicit user/workspace sharing rights. Ownership and grants determine which user can view/edit/manage a report.

• execution_runs, background_jobs, and job_attempts: run/job status and retries for generated reports.

• Object/file storage: the actual generated PDF/CSV artifact; its URI is recorded in reports.artifact_uri.

Foreign-key direction matters: reports.execution_run_id references execution_runs.execution_run_id; the relationship is not represented by inventing a report_id FK on execution_runs. Likewise, report_data_snapshots, report_citations, and report_access_grants each relate to reports; they are not a serial chain through one another.

7.4 Access and provenance expectations

The reporting API must check data access as well as report access. Sharing a report must not bypass rules governing private source documents or underlying datasets. Reports should retain as-of/source snapshot metadata and citations so a reopened report can explain the data and evidence behind its claims. PDF/CSV exports are artifacts; PostgreSQL records the report metadata and lineage.

8. Image 7 - Portfolio Management and Analytics

Image file: FinSight AI Portfolio Analytics Diagram(1).png
Primary purpose: Explain how user portfolio ledgers and point-in-time holdings are combined with market prices and benchmarks to produce performance, allocation, attribution and risk insights for the dashboard and reports.

8.1 Client and backend responsibilities

The client exposes a Portfolio Dashboard, Portfolio Management UI and Analytics/Insights UI. Users can create/manage portfolios, record transactions, manage cash/holdings/watchlists, view valuation/performance, examine allocation and attribution, compare with benchmarks, view risk measures and generate reports.

The backend has distinct logical responsibilities:

- **Portfolio Service:** Portfolio identity/settings, saved portfolio state and selected snapshot.
- **Transaction Service:** Activity ledger for transactions, fees, cash flows and applicable corporate-action adjustments.
- **Market Data Service:** Historical price data, provider selection, freshness, adjustment handling, and instrument metadata.
- **Analytics Service:** Performance/return calculations, benchmark comparison, attribution, allocation, and preparation of data for charts/reports.
- **Risk Analytics Service:** Invokes the risk domain for VaR/CVaR, drawdown, Monte Carlo, correlation and stress analytics; it should not duplicate a second risk calculation implementation within the portfolio module.
- **Report Generation Service:** Builds saved portfolio/performance/risk reports and retains source snapshots.

8.2 Portfolio persistence model

- portfolios represents the user/workspace portfolio container, base currency, benchmark and settings.
- portfolio_transactions stores the activity ledger (transaction type/time, instrument if applicable, quantity, price/amount, fees, currency, data snapshot and creator as applicable).
- portfolio_snapshots represents the portfolio's point-in-time valuation and records its data_snapshot_id, as_of_at, total market/equity/cash values, currency, valuation status and limitations.
- portfolio_positions stores holdings/weights/cost/P&L as of a specific portfolio_snapshot_id and links to canonical instruments.
- portfolio_cash_balances stores cash balances per snapshot and currency; its key is (portfolio_snapshot_id, currency_code).
- portfolio_returns stores daily/period returns and benchmark returns, associated with the portfolio and applicable data snapshot.
- Related entities include watchlists, benchmarks/instruments, price_bars, corporate actions, and data snapshots.

The relational arrows are separate FK relationships: portfolios own transactions and snapshots; snapshots own positions and cash balances; returns refer to portfolios and input snapshots; market data is a separate source for valuation. The diagram should not be read as transactions being the parent of snapshots or position rows being the parent of price bars.

8.3 Analytics behavior

The Performance Engine computes total/annualized return, time-weighted or money-weighted return when supported, and drawdown/volatility. Benchmark Comparison evaluates absolute/relative performance and tracking/risk metrics where supported. Attribution estimates contributions from asset/security selection and allocation where the inputs support them. Allocation Analysis groups the portfolio by asset class, sector/industry, geography or other configured classifications. Risk Analytics calls the quantitative risk component.

The dashboard shows an as-of date/time, data mode and freshness status such as historical, latest available, fresh, cached or stale. In the initial release, use historical daily data and clearly label the available source coverage. Intraday/real-time data is conditional on provider support and entitlement; no live broker execution is part of Release 1.

8.4 RAG separation

The chunk_embeddings/pgvector component is for research document retrieval and RAG. It is not a dependency for ordinary portfolio calculation. The portfolio service should query structured holdings, price/fundamental data and stored analytics results. If a future semantic-retrieval use case is added to portfolio analytics, describe it separately rather than implying all calculations depend on pgvector.

---

9. Image 8 - Paper Trading and Broker Integration

Image file: FinSight AI Trading Architecture Diagram(1).png
Primary purpose: Describe a future, separately authorised extension for simulated trading and optional broker order execution. This image is not a Release 1 implementation requirement.

9.1 Release 2 boundary

The SRS includes broker settings, paper trading and execution monitoring as conditional capabilities, but the database package explicitly separates the broker/trading extension. Do not apply schema_optional_trading.sql, expose broker order endpoints, or make live trading a dependency of Release 1. Keep this diagram labelled **Future Extension - Release 2**.

9.2 Frontend components

- **Trading UI:** Paper trading dashboard, account/mode selection, simulated order entry, order status and trade history.
- **Account & Trading Settings:** Manage eligible broker connections, permissions, simulation settings and allowed safety limits without showing stored secrets.
- **Portfolio & Analytics UI:** View positions, cash, transaction and P&L history, risk exposure, benchmark comparisons and as-of/freshness information.

The UI only submits order requests to the authenticated backend. It never sends an order directly to a provider from the browser or trusts client-side risk checks.

9.3 Backend components and order flow

1. Broker Connection Service manages provider selection, secure connection references, connection tests and account synchronisation.
2. Order Management validates and stores order intent and lifecycle state.

3. Pre-Trade Risk Check Service evaluates configured order/position/exposure constraints and records results.

4. Approval & Confirmation requires the applicable user confirmation or approval for sensitive/live orders.

5. Execution Service sends approved requests to a simulator or broker adapter, tracks acknowledgements, rejections, cancels, fills and errors, and updates portfolio/account state.

6. Reconciliation & Sync compares local order/account/position state with provider state and records discrepancies for resolution.

7. Monitoring/notifications and the trade journal/report layer expose status, risk alerts, reconciliation exceptions and post-trade notes.

Route provider API traffic through a backend broker adapter/provider interface rather than drawing an external broker as an ordinary frontend caller of the API gateway.

9.4 Canonical optional trading tables

The optional schema contains 15 tables. Their responsibilities are:

- Provider/connection/account: broker_providers, broker_connections, broker_accounts, paper_trading_accounts.
- Safety policy and orders: trading_safety_policies, trading_orders, trading_risk_checks, trading_order_approvals.
- Order lifecycle/fills: trading_order_events, trading_order_fills.
- Broker positions and reconciliation: broker_position_snapshots, broker_positions, broker_reconciliations, broker_reconciliation_items.
- Trade journal: trade_journal_entries.

The paper account is linked to a core portfolios record. The broker connection stores an external secret reference such as secret_ref; it must not store a plaintext API secret. Broker position snapshots are point-in-time provider state and are distinct from locally managed portfolio snapshots.

9.5 Safety rules

- Live order capability is disabled by default.
- Read-only market/account access, paper simulation, and live order permission are distinct modes.
- Orders should fail closed when required authorization, pre-trade checks, approval, market/account state, or provider response is unavailable or unresolved.
- Live orders require the configured explicit confirmation and other applicable safety controls.
- Order requests need idempotency; order events/fills preserve lifecycle and external references; reconciliation records mismatches rather than silently overwriting them.
- Analysis, screening, signals, DCF and backtesting do not themselves place an order. Any future order passes through the separately authorized trading workflow.

10. Capabilities represented inside the diagrams but without their own standalone image

The current set has eight images. A few logical domains are represented within the overall architecture or an adjacent diagram rather than as a dedicated component diagram.

10.1 DCF Valuation Studio

The Overall System Architecture includes a DCF Valuation Service, but there is no separate DCF image in this set. Interpret the service as a deterministic Python calculation component. It obtains selected historical financial data and its data_snapshot_id, loads a versioned valuation model, accepts explicit assumptions, computes forecasts and free cash flow, applies WACC and terminal-value assumptions, derives enterprise/equity/per-share value, and generates scenario/sensitivity results. It saves an immutable or traceable valuation run with the assumptions, data/model versions, calculated forecast values and warnings/limitations.

Relevant schema entities include valuation_model_versions, valuation_runs, valuation_assumptions, valuation_forecasts, valuation_sensitivity_runs and associated sensitivity/result/comparison tables. Each assumption should retain its value, unit/period, source/user origin and evidence/provenance where defined by the schema. The language model can explain the method and results but should not replace the deterministic calculator.

The schema README identifies detailed DCF scope as added analytical design detail beyond the broad SRS. Before formal test sign-off, represent detailed valuation requirements in the SRS with explicit requirement IDs (for example a FR-VAL-* series) and map each to tests.

10.2 Authentication, Authorization and Security

The overall image includes an Auth Service and the report/document/trading diagrams include resource-level authorization. The security domain is cross-cutting. It includes users and workspaces, roles and permissions, sessions, workspace membership, ownership, resource-specific access grants, audit records, secret references, and step-up/confirmation checks where required. Backend authorization must be applied to every protected endpoint. RAG permission filtering applies before context construction. Source connection credentials must not appear in prompts, normal API responses, or ordinary logs.

A separate security diagram could later expand these controls, but its absence does not mean the controls are optional.

10.3 Background Jobs and Observability

Background jobs are shown in the overall architecture and in data ingestion, document processing, risk, strategy/backtesting and reporting. These views describe the same shared operational capability: schedule or dispatch jobs, track queued/running/succeeded/failed/cancelled state, retain attempts/errors, retry safely, enforce idempotency, record execution steps/events, expose health checks, and notify users about relevant outcomes.

Canonical operational entities include background_jobs, job_attempts, execution_runs, execution_steps, execution_events, and service_health_checks. The diagram boxes do not imply that every domain needs its own independent scheduler/queue. A shared worker system can serve many job types.

10.4 Financial Data Explorer

The Data Explorer is a named SRS page/tool domain but does not have its own standalone image in the current set. Its responsibilities include approved structured-source registration, schema discovery, access allowlists, controlled search/querying, read-only-by-default model-generated queries, query validation,

timeouts/row limits, transformations, dataset joins, saved queries/datasets, exports and lineage. Relevant schema entities include structured_data_sources, saved_queries, query_executions, user datasets and transformation/lineage tables.

The Overall System Architecture's tool interface and Market Data Service should not be treated as a complete visual specification of every Data Explorer function.

---

11. Cross-diagram end-to-end flows

11.1 From market provider to saved financial answer

Provider adapter → ingestion batch/raw records → normalize and quality validation → create or update data_snapshots → persist price_bars/fundamentals/corporate actions → DCF/risk/portfolio/screening/backtesting service queries the versioned data → AI Analyst calls the approved tool where appropriate → result is returned with as-of/snapshot/method/assumptions → report service saves inputs and citations when the user asks to save/export.

11.2 From uploaded filing to RAG answer

Upload/source connector → file safety and duplicate check → object storage + research_documents metadata → document_versions and extraction → tables/chunks → chunk_embeddings linked to knowledge_indexes → indexing status/reconciliation → query embedding → access-filtered search → retrieval_runs and retrieval_run_chunks trace → authorized source context passed to Ollama → answer with citations → optional report saves the citation and input provenance.

11.3 From saved strategy to reproducible backtest report

User definition → strategies and immutable strategy_versions → readiness validation → select input data_snapshot_id, optional strategy variant and cost model → create execution_runs/backtest_runs → compute trades, equity points and metrics → save result/limitations → report assembly records snapshot lineage and citations if applicable → export worker creates artifact → reports and job status allow reopen/download.

11.4 From portfolio snapshot to risk results

Read portfolio ledger and market data → create/select portfolio_snapshots with data_snapshot_id → freeze input positions in risk_run_positions → record risk_runs configuration/model versions/period/seed → parallel metric/simulation/stress calculations → persist metric/time-series/distribution/stress outputs → aggregate charts and warnings → optionally save a report tied to its input snapshots.

11.5 Future broker order flow (Release 2 only)

Authorized order request → order validation → pre-trade risk checks → required explicit approval/confirmation → idempotent submission to a provider adapter or simulator → order events/fills/status → account/position snapshot update → reconciliation → audit/journal/report. If a blocking safety check or required provider response is unavailable, do not continue as if the order succeeded.

---

12. Database interpretation and naming rules for AI/developers

1. **Canonical SQL wins.** When a picture uses a shorthand label, map it to the nearest declared table only when the purpose genuinely matches. Do not assume a field exists just because the diagram displays it.

2. **Do not create new tables from logical components.** An Indicator Calculation Service, Risk Results Aggregation, or Report Assembly Service is a service/component, not automatically a physical table.

3. **Respect composite keys.** Examples include screen_results keyed by (screen_run_id, instrument_id), risk_metric_results keyed by (risk_run_id, metric_key), portfolio_cash_balances keyed by (portfolio_snapshot_id, currency_code), backtest_equity_points keyed by (backtest_run_id, observation_date), and report_data_snapshots keyed by (report_id, data_snapshot_id).

4. **Respect FK direction.** In the schema, a child FK points to its parent. Avoid drawing an adjacent-row sequence as if every table were a child of the previous one. Use relationship lines only when the FK or logical service call actually exists.

5. **Persist input lineage for analysis.** A DCF/risk/screen/backtest/report should retain the appropriate source snapshot(s) plus configuration/model/strategy version, parameters, and status/limitations where supported.

6. **Separate raw, normalized and derived data.** Raw provider payloads, normalized bars/facts, document text/chunks, embeddings, calculated metrics, and report artifacts have different roles and retention expectations.

7. **Separate external storage from relational metadata.** Object/file storage contains original document and generated artifact bytes. PostgreSQL stores metadata and references to those files.

8. **Treat pgvector as part of PostgreSQL for the initial release.** Its use is for similarity retrieval, linked to stable document chunk and index IDs. It is not a separate source of financial truth.

9. **Treat reports, retrieval traces, and execution traces as first-class lineage.** A final answer/report should be auditable where supported, but logs should not leak secrets or unnecessary private text.

10. **Do not implement Release 2 tables in Release 1.** The broker/paper-trading DDL is optional and needs its own integration, security and end-to-end safety validation.

---

13. Release and implementation interpretation

Release 1A - Analytical data foundation

Prioritise identity/workspaces; provider adapters and credentials via secure secret references; US-equity instrument master; historical daily prices, financial statements/facts and corporate actions; data snapshots; ingestion/quality; data exploration; basic portfolio/watchlists; dashboard and freshness labels.

Release 1B - Financial research and AI

Add document/source registration, file validation, extraction, versioning, chunks, pgvector embeddings/indexes, access-filtered retrieval, conversation/message persistence, model/configuration records, approved tool calls, and source-cited answers/reports.

Release 1C - DCF and portfolio risk

Add valuation model versions and assumptions/provenance, forecasts and sensitivity runs; portfolio snapshots/returns; risk model/config versions; frozen risk positions; VaR/CVaR, drawdown, covariance, stress and seeded Monte Carlo outputs. The DCF/risk scope should be explicitly represented in the formal SRS before contractual acceptance testing.

Release 1D - Research workflows and backtesting

Add screen runs/results, indicator definitions/profiles, strategy versions/variants, signal evidence/status, cost model versions, reproducible backtests, trades/equity curves/metrics, comparison groups and scheduled jobs.

Release 2+ - Paper trading and optional broker integration

Only after separate readiness/security testing: broker adapters, paper accounts, orders, fills, approval gates, pre-trade risk, reconciliation, order/audit events and trade journaling. Live order operations should default to disabled, require explicit authorisation/confirmation, and fail closed.

---

14. Scope points and limitations to preserve

- The initial market focus is US equities. India support is intended through more provider/exchange/calendar configuration after the first market pipeline has been validated; do not imply that every provider covers every market equally.
- Historical daily data is the Release 1 baseline. Intraday/live data is conditional on source, entitlement and scope settings.
- No standalone DCF, dedicated security, or dedicated background-jobs image is included in the current eight-image set. These responsibilities are described in the overall architecture and this companion guide.
- DCF and detailed quantitative-risk specifications are present in the database design/added scope notes, but the schema package itself recommends adding explicit FR-VAL-* and FR-RISK-* requirements to the formal SRS before final requirement-to-test sign-off.
- The diagrams are logical component views and may simplify schema fields. They must not be used instead of reading schema.sql, schema_optional_trading.sql, and table_catalog.md when implementing migrations or queries.
- Nothing in this document claims a runtime database migration, provider integration, risk-method validation, or end-to-end application test has already passed. Those must be verified during implementation.

---

15. Source documents in the project folder

The source baseline used here is FinSight_Software Requirements Specification (SRS)(3).pdf and FinSight_AI_Database_Schema_and_ERD_Reference(1).pdf. The canonical implementation references are FinSight_DB_Design/schema.sql, FinSight_DB_Design/schema_optional_trading.sql, FinSight_DB_Design/table_catalog.md, and FinSight_DB_Design/README.md.

If duplicate SRS or database-reference files are present, use the version explicitly chosen as the project's current baseline and keep it consistent across diagram review, implementation, and testing.